

Desenvolvimento de Plugins QGIS com Python

Do básico ao avançado, com GPX Batch Converter e GeoClick Capture

Jubílio Filiano Mause

24 de Julho de 2026

Índice geral

Parte I - Fundamentos

O que é um plugin QGIS?

Ambiente de desenvolvimento

Planeamento e arquitectura

Parte II - O primeiro plugin

Estrutura mínima do pacote

Ações, menus e barra de ferramentas

Interfaces: QDialog e QDockWidget

Validação de entradas e experiência do utilizador

Parte III - Desenvolvimento intermédio

Trabalhar com projectos e camadas

Sistemas de referência e coordenadas

Leitura de GPX e escrita de formatos GIS

Tarefas em segundo plano

Resultados, progresso e logs

Ferramentas de mapa

Snapping a vértices e segmentos

Pedidos de rede e geocodificação reversa

Preferências persistentes

Parte IV - Desenvolvimento avançado

Compatibilidade QGIS 3/4 e Qt 5/6

Segurança

Testes

Empacotamento

Git e fluxo de colaboração

Integração contínua e releases

Publicação no repositório oficial do QGIS

Tradução, acessibilidade e documentação

Parte V - Estudos de caso

GPX Batch Converter

GeoClick Capture

Comparação dos dois padrões

Parte VI - Projecto prático

Construir o Quick Point Logger

Exercícios graduais

Diagnóstico de erros

Checklists

Próximos níveis

Apêndices: metadata, unload, resultados de lote, glossário e referências.

Prefácio

Este manual foi concebido para quem utiliza QGIS e pretende passar de utilizador avançado a criador de ferramentas reutilizáveis. O objectivo não é apenas mostrar como colocar um botão no QGIS. O foco é ensinar um processo completo: identificar um problema, desenhar uma arquitectura simples, desenvolver uma interface segura, trabalhar com dados geográficos, manter compatibilidade entre versões, testar, empacotar, publicar e evoluir o plugin sem perder qualidade.

Os exemplos centrais são dois plugins desenvolvidos a partir de necessidades reais:

GPX Batch Converter, que transforma lotes de ficheiros GPX em formatos GIS, suporta fusão, execução em segundo plano, cancelamento, relatórios e validações de segurança;

GeoClick Capture, que regista cliques no mapa como pontos auditáveis, aplica snapping, transforma coordenadas, identifica feições, efectua geocodificação reversa opcional e organiza sessões num painel lateral.

Os dois estudos de caso foram escolhidos porque representam famílias diferentes de plugins. O primeiro é orientado a processamento de ficheiros e tarefas demoradas. O segundo é orientado a interacção com o mapa, edição, rede e controlo de estado. Em conjunto, cobrem grande parte dos padrões encontrados no desenvolvimento profissional de extensões QGIS.

Este texto usa **Python**, **PyQGIS** e **qgis.PyQt**. As práticas são pensadas para QGIS 3.28 ou superior e para a transição para QGIS 4 e Qt 6. A API evolui; por isso, antes de adoptar um método específico num projecto de longo prazo, consulte sempre a documentação da versão de QGIS que será suportada.

Como utilizar o manual

O manual está organizado em seis partes:

Fundamentos - arquitectura, ambiente e planeamento;

Primeiro plugin - estrutura mínima, acções e interface;

Nível intermédio - camadas, CRS, ficheiros, tarefas, mapa e definições;

Nível avançado - Qt 5/6, segurança, testes, CI/CD e publicação;

Estudos de caso - decisões reais dos dois plugins;

Projecto prático - construção de um plugin completo e exercícios.

Pode seguir os capítulos em sequência ou usar o manual como referência. Os blocos **Prática recomendada**, **Erro comum** e **Decisão de arquitectura** destacam os pontos que normalmente causam falhas em produção.

Roteiro de aprendizagem

Ao concluir este manual, o leitor deverá ser capaz de:

distinguir um script da consola Python, um algoritmo Processing e um plugin completo;

estruturar um pacote instalável pelo gestor de plugins do QGIS;
 implementar correctamente `classFactory()`, `initGui()`, `run()` e `unload()`;
 criar acções, menus, barras de ferramentas, diálogos e painéis laterais;
 utilizar widgets QGIS, como `QgsMapLayerComboBox`;
 ler e escrever camadas vectoriais e transformar coordenadas entre CRS;
 executar tarefas demoradas sem bloquear a interface;
 adicionar cancelamento, progresso, logs e resultados estruturados;
 criar ferramentas de clique e snapping no mapa;
 efectuar pedidos de rede através de `QgsNetworkAccessManager`;
 persistir preferências com `QgsSettings`;
 manter compatibilidade entre Qt 5 e Qt 6;
 reduzir riscos relacionados com subprocessos, caminhos e serviços externos;
 criar testes unitários e validações de pacote;
 automatizar releases e preparar o plugin para o repositório oficial do QGIS.



Ciclo completo de desenvolvimento

Parte I - Fundamentos

1. O que é um plugin QGIS?

Um plugin é um pacote de software carregado pelo QGIS para acrescentar comportamentos que não existem no núcleo da aplicação ou que precisam de ser adaptados a um fluxo específico. Um plugin pode ser pequeno, com uma única acção, ou incluir interfaces, algoritmos, painéis, serviços de rede, gestão de dados e automação.

Python é a opção mais acessível para a maioria dos plugins QGIS porque não exige compilação separada para Windows, Linux e macOS. O pacote contém ficheiros Python, metadata, documentação e recursos. O gestor de plugins instala o ZIP e o QGIS importa o pacote no perfil do utilizador.

1.1 Script, algoritmo Processing ou plugin?

Use a **consola Python** quando:

pretende testar uma ideia rapidamente;
 o código será executado poucas vezes;
 não precisa de distribuição nem de interface estável.

Use um **algoritmo Processing** quando:

a função recebe dados, parâmetros e produz resultados;
o fluxo deve funcionar em lote, no modelador ou por linha de comandos;
não precisa de interacção contínua com o mapa.

Use um **plugin completo** quando:

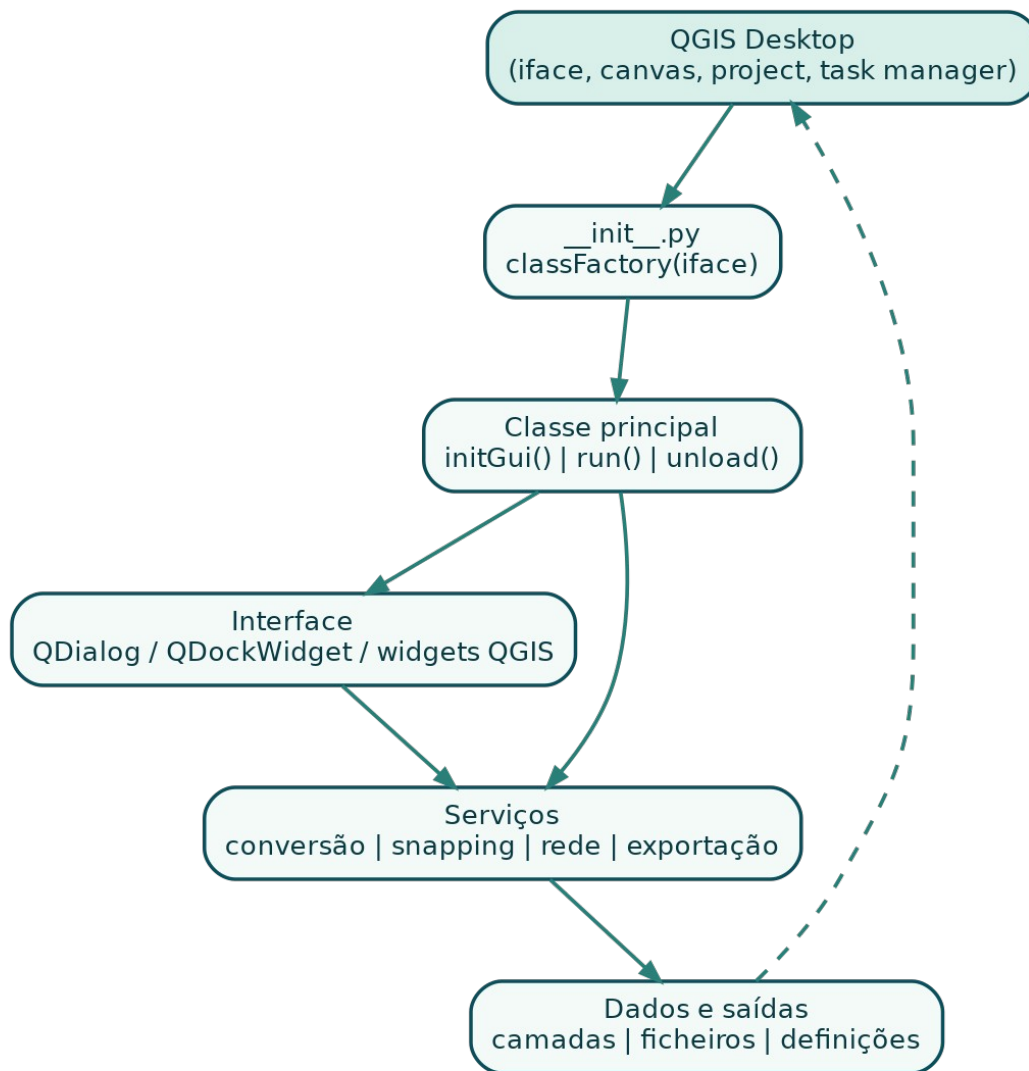
precisa de menu, barra de ferramentas, diálogo ou painel;
precisa de manter estado entre acções;
reage a cliques, mudanças de projecto ou selecção de camadas;
integra tarefas, rede, edição ou múltiplos serviços.

O GPX Batch Converter poderia, em parte, ser um algoritmo Processing. Contudo, a necessidade de resultados tabulares, cancelamento, selecção de formatos, relatórios e gestão de tarefas justificou uma interface própria. O GeoClick Capture necessita de um plugin completo porque mantém uma ferramenta de mapa activa, um painel de sessão e preferências persistentes.

1.2 O contrato entre QGIS e o plugin

O QGIS não adivinha como iniciar o código. Ele procura uma pasta válida, lê metadata.txt, importa `__init__.py` e chama `classFactory(iface)`. Esta função devolve uma instância da classe principal. Depois, o QGIS chama `initGui()` para registar a interface e `unload()` quando o plugin é desactivado.

O objecto `iface` é a principal porta de entrada para a interface do QGIS. Através dele, o plugin acede ao mapa, à janela principal, aos menus, à barra de mensagens e a outros componentes.



Arquitectura simplificada

1.3 Princípio fundamental: resolver um problema definido

Antes de escrever código, responda:

Quem utilizará o plugin?

Qual é a tarefa repetitiva ou propensa a erro?

Quais entradas e saídas são necessárias?

O QGIS já possui a funcionalidade?

Existe um plugin semelhante?

O plugin deve funcionar sem Internet?

Que versões de QGIS e sistemas operativos serão suportadas?

Como o utilizador saberá que a operação terminou ou falhou?

Um escopo bem definido evita um plugin que tenta fazer tudo e não executa nada com clareza.

Estudo de caso - diferenciação: o projecto inicialmente chamado QGIS LatLon sobrepunha-se a ferramentas existentes de coordenadas. O reposicionamento para **GeoClick Capture** definiu um propósito distinto: criar registos auditáveis de cliques, com sessão, metadados, snapping e revisão.

2. Ambiente de desenvolvimento

2.1 Instalações recomendadas

Prepare:

QGIS suportado pelo projecto;

editor como Visual Studio Code, PyCharm ou outro com suporte Python;

Git;

Plugin Reloader, para recarregar o plugin durante o desenvolvimento;

opcionalmente Qt Designer, para interfaces .ui;

uma pasta de dados mínimos de teste.

Ao suportar QGIS 3 e 4, mantenha pelo menos um ambiente de teste para cada geração. Testar apenas no QGIS instalado no computador do autor é insuficiente.

2.2 Pasta de plugins do perfil

Em Windows, as localizações mais comuns são:

C:\Users\<UTILIZADOR>\AppData\Roaming\QGIS\QGIS3\profiles\default\python\plugins\

C:\Users\<UTILIZADOR>\AppData\Roaming\QGIS\QGIS4\profiles\default\python\plugins\

Em Linux:

~/local/share/QGIS/QGIS3/profiles/default/python/plugins/

~/local/share/QGIS/QGIS4/profiles/default/python/plugins/

O nome da pasta deve ser um identificador Python válido. Use letras minúsculas, números e _.
Não use hífen.

correcto: quick_point_logger

incorrecto: quick-point-logger

2.3 Instalação de desenvolvimento

Pode copiar a pasta ou cloná-la directamente no perfil:

`git clone https://github.com/UTILIZADOR/REPOSITORIO.git quick_point_logger`

Durante o desenvolvimento:

altere o código no editor;

guarde os ficheiros;

recarregue o plugin;

teste o fluxo afectado;
consulte o painel **Log Messages** e a consola Python.

2.4 Organização do repositório

Uma estrutura prática é:

```
project-root/
├── .github/workflows/
├── docs/
├── sample_data/
├── tests/
├── quick_point_logger/
│   ├── __init__.py
│   ├── metadata.txt
│   ├── plugin.py
│   ├── dialog.py
│   ├── icons/
│   ├── LICENSE
│   └── README.md
├── CHANGELOG.md
├── LICENSE
└── README.md
```

O ZIP instalável deve conter apenas a pasta do plugin como directório de topo. O repositório pode ter testes e documentação adicionais fora dessa pasta.

3. Planeamento e arquitectura

3.1 Escrever uma especificação curta

Antes do código, escreva uma página com:

problema;
utilizadores;
funcionalidades obrigatórias;
funcionalidades futuras;
dados de entrada e saída;
dependências;
restrições de segurança;
critérios de aceitação.

Exemplo para o GPX Batch Converter:

Problema: converter centenas de GPX manualmente é demorado e inconsistente.

Entradas: pasta com .gpx, tipos de subcamada, formato de saída.

Saídas: ficheiros GIS, relatório de resultados e mensagens de erro.

Critérios: interface responsiva, cancelamento, nomes seguros, sem shell=True.

3.2 Separar interface, lógica e infra-estrutura

Evite colocar toda a aplicação num único método de botão. Uma divisão mínima:

classe principal: integração com QGIS;

diálogo/painel: widgets e sinais;

serviço ou tarefa: processamento;

utilitários: validação, nomes, compatibilidade;

testes: funções independentes e regras de pacote.

A separação melhora testes, manutenção e compatibilidade. No GPX Batch Converter, a classe principal regista a acção, o diálogo recolhe opções e GpxConversionTask executa o lote. No GeoClick Capture, a classe principal gere o mapa, enquanto o painel gere a sessão e a tabela.

3.3 Estado do plugin

Identifique o estado que precisa de ser mantido:

acções e widgets;

ferramenta de mapa activa;

camada de destino;

tarefa em execução;

pedidos de rede pendentes;

última coordenada;

preferências;

resultados da última operação.

Todo recurso criado em initGui() deve ser removido, fechado ou cancelado em unload().

Parte II - O primeiro plugin

4. Estrutura mínima do pacote

4.1 metadata.txt

O metadata.txt descreve o plugin ao QGIS e ao repositório oficial.

[general]

name=Quick Point Logger

description=Captures map clicks into a temporary point layer.

version=0.1.0

```

qgisMinimumVersion=3.28
qgisMaximumVersion=4.99
author=Seu Nome
email=nome@example.org
homepage=https://github.com/user/quick-point-logger#readme
repository=https://github.com/user/quick-point-logger
tracker=https://github.com/user/quick-point-logger/issues
license=MIT
category=Vector
tags=point,capture,coordinates,logging
experimental=True
deprecated=False
hasProcessingProvider=False
supportsQt6=True
icon=icons/capture.svg

```

Campos essenciais

name: nome público, estável e distinto;

description: uma frase curta;

about: explicação detalhada, dependências e limitações;

version: versão SemVer;

qgisMinimumVersion e qgisMaximumVersion;

repository, tracker, homepage e license;

supportsQt6=True quando validado para Qt 6.

O repositório oficial exige documentação mínima, links funcionais, licença compatível, descrição em inglês, ausência de binários e um pacote dentro do limite de tamanho aplicável.

4.2 __init__.py

```

def classFactory(iface):
    """Return the plugin instance used by QGIS."""
    from .plugin import QuickPointLoggerPlugin
    return QuickPointLoggerPlugin(iface)

```

Mantenha esta função pequena. A importação dentro de classFactory() reduz efeitos colaterais durante a descoberta do plugin.

4.3 Classe principal

```

from pathlib import Path

from qgis.PyQt.QtGui import QIcon
from qgis.PyQt.QtWidgets import QAction

```

```

class QuickPointLoggerPlugin:
    MENU = "&Quick Point Logger"

    def __init__(self, iface):
        self.iface = iface
        self.action = None
        self.icon_path = Path(__file__).parent / "icons" / "capture.svg"

    def initGui(self):
        self.action = QAction(
            QIcon(str(self.icon_path)),
            "Quick Point Logger",
            self.iface.mainWindow(),
        )
        self.action.triggered.connect(self.run)
        self.iface.addPluginToVectorMenu(self.MENU, self.action)
        self.iface.addToolBarIcon(self.action)

    def run(self):
        self.iface.messageBar().pushInfo(
            "Quick Point Logger",
            "The plugin is running.",
        )

    def unload(self):
        if self.action is None:
            return
        self.iface.removePluginVectorMenu(self.MENU, self.action)
        self.iface.removeToolBarIcon(self.action)
        self.action.deleteLater()
        self.action = None

```

Este padrão aparece no GPX Batch Converter: `initGui()` cria `QAction`, associa `triggered` ao método `run()`, adiciona ao menu Vector e à barra; `unload()` remove os elementos e fecha a interface.

4.4 Ciclo de vida e limpeza

Um plugin que funciona ao activar, mas deixa sinais ou tarefas depois de desactivar, é instável. No `unload()`:

- desassocie ou remova acções;
- cancele tarefas activas;
- aborte respostas de rede;
- remova painéis;
- desactive ferramentas de mapa;

chame `deleteLater()` para widgets Qt;
 limpe referências Python.

5. Acções, menus e barra de ferramentas

`QAction` representa uma operação reutilizável. A mesma acção pode estar no menu e na barra.

```
self.action = QAction(QIcon(icon_path), "Converter GPX", parent)
self.action.setObjectName("gpxBatchConverterAction")
self.action.setStatusTip("Converter vários ficheiros GPX")
self.action.triggered.connect(self.run)
```

Escolha o menu por domínio:

Vector para operações vectoriais;

Raster para raster;

Web para serviços Web;

Database para bases de dados.

Não crie um menu de topo desnecessário para uma única acção.

5.1 Acções verificáveis

Para ligar e desligar modos:

```
self.capture_action.setCheckable(True)
self.capture_action.toggled.connect(self.activate_capture)
```

Sincronize o estado entre toolbar, menu e painel. Ao mudar programaticamente, bloqueie sinais para evitar ciclos:

```
self.capture_action.blockSignals(True)
self.capture_action.setChecked(enabled)
self.capture_action.blockSignals(False)
```

5.2 Ícones

Use SVG simples ou PNG otimizado. O ícone deve permanecer legível em 16, 24 e 32 pixels.
 Use caminhos relativos ao pacote:

```
ICON_DIR = Path(__file__).parent / "icons"
QIcon(str(ICON_DIR / "capture.svg"))
```

GeoClick Capture 1.2.6 usa ícones separados para captura, log, snapping, geocodificação, exportação, undo, eliminação e sessões. A distinção visual reduz erros operacionais.

6. Interfaces: QDialog e QDockWidget

6.1 Quando usar cada um

Use QDialog para uma tarefa com início e fim, como converter ficheiros. Use QDockWidget para uma ferramenta que acompanha o trabalho no mapa, como um log de captura.

GPX Batch Converter: diálogo com pastas, formatos, opções, progresso e resultados.

GeoClick Capture: painel lateral com sessão, camada, snapping, tabela e exportação.

6.2 Interface programática ou Qt Designer

Interface programática:

facilita compatibilidade e revisão de código;

não exige compilar .ui;

funciona bem para formulários médios.

Qt Designer:

acelera layouts complexos;

separa desenho visual;

exige estratégia para carregar .ui sem ficheiros gerados desnecessários.

O repositório oficial recomenda evitar ficheiros gerados como ui_*.py quando não são necessários. Pode carregar o .ui em tempo de execução.

6.3 Um selector de pasta reutilizável

```
class FolderSelector(QWidget):
```

```
    def __init__(self, title, parent=None):
```

```
        super().__init__(parent)
```

```
        self.title = title
```

```
        self.path_edit = QLineEdit()
```

```
        self.browse_button = QPushButton("Browse...")
```

```
        self.browse_button.clicked.connect(self.browse)
```

```
        layout = QHBoxLayout(self)
```

```
        layout.setContentsMargins(0, 0, 0, 0)
```

```
        layout.addWidget(self.path_edit, 1)
```

```
        layout.addWidget(self.browse_button)
```

```
    def browse(self):
```

```
        selected = QFileDialog.getExistingDirectory(self, self.title)
```

```
        if selected:
```

```
            self.path_edit.setText(selected)
```

Transformar um conjunto repetido de widgets numa classe reduz duplicação e facilita validação.

6.4 Widgets QGIS

Sempre que possível, use widgets que conhecem o projecto. Exemplo:

```
from qgis.core import Qgis, QgsMapLayerProxyModel
from qgis.gui import QgsMapLayerComboBox

self.layer_combo = QgsMapLayerComboBox()
point_filter = getattr(QgsMapLayerProxyModel, "PointLayer", None)
if point_filter is None:
    point_filter = Qgis.LayerFilter.PointLayer
self.layer_combo.setFilters(point_filter)
self.layer_combo.setAllowEmptyLayer(True)
self.layer_combo.setShowCrs(True)
```

QgsMapLayerComboBox acompanha camadas adicionadas, removidas e renomeadas. Uma QComboBox manual torna-se rapidamente desactualizada.

7. Validação de entradas e experiência do utilizador

Valide antes de iniciar trabalho pesado:

pasta de entrada existe;

pasta de saída pode ser criada;

existe pelo menos um ficheiro compatível;

pelo menos uma camada ou opção foi seleccionada;

executáveis necessários foram encontrados;

camada de destino é válida e tem geometria adequada;

caminhos não são iguais quando isso causa sobrescrita perigosa.

Mensagens de erro devem indicar acção correctiva:

Fraco: Conversion failed.

Melhor: No .gpx files were found in the selected input folder.

Evite caixas modais para cada aviso dentro de um lote. Use log, tabela de resultados e resumo final.

Parte III - Desenvolvimento intermédio

8. Trabalhar com projectos e camadas

8.1 QgsProject

A instância actual:

```
project = QgsProject.instance()
```

Adicionar camada:

```
layer = QgsVectorLayer(path, layer_name, "ogr")
if layer.isValid():
    project.addMapLayer(layer)
```

Nunca assuma que a camada carregou. Valide e registe a mensagem do fornecedor quando disponível.

8.2 Camada de memória

GeoClick Capture cria uma camada temporária quando não existe destino:

```
layer = QgsVectorLayer(
    "Point?crs=EPSG:4326",
    "Captured Points Log",
    "memory",
)
provider = layer.dataProvider()
provider.addAttributes([
    QgsField("id", QVariant.Int),
    QgsField("captured_at", QVariant.String),
    QgsField("lat", QVariant.Double),
    QgsField("lon", QVariant.Double),
])
layer.updateFields()
QgsProject.instance().addMapLayer(layer)
```

Uma camada de memória é útil para sessões temporárias. Para persistência imediata, prefira GeoPackage.

8.3 Adicionar feições

```
feature = QgsFeature(layer.fields())
feature.setGeometry(QgsGeometry.fromPointXY(point))
feature["id"] = next_id
feature["captured_at"] = timestamp
feature["lat"] = latitude
feature["lon"] = longitude
```



```
ok, created = layer.dataProvider().addFeatures([feature])
```

if not ok:

```
    raise RuntimeError("The point could not be added.")
```

Considere edição transaccional e undo/redo quando altera camadas existentes do utilizador. Para camadas geridas pelo plugin, a escrita directa pelo provider pode ser suficiente, desde que os IDs e campos sejam controlados.

9. Sistemas de referência e coordenadas

9.1 Nunca confundir coordenadas do mapa com latitude/longitude

O clique é devolvido no CRS do mapa. Se o projecto estiver em UTM, os valores podem ser Este/Norte em metros, não longitude/latitude.

```
project_crs = canvas.mapSettings().destinationCrs()
wgs84 = QgsCoordinateReferenceSystem("EPSG:4326")
transform = QgsCoordinateTransform(
    project_crs,
    wgs84,
    QgsProject.instance(),
)
wgs84_point = transform.transform(map_point)
lon = float(wgs84_point.x())
lat = float(wgs84_point.y())
```

9.2 Transformar para a camada de destino

Se a camada de destino tem outro CRS, transforme a geometria antes de gravar:

```
layer_transform = QgsCoordinateTransform(
    project_crs,
    layer.crs(),
    QgsProject.instance(),
)
layer_point = layer_transform.transform(map_point)
```

9.3 Guardar ambos os sistemas

Para auditoria, é útil guardar:

latitude e longitude em WGS 84;

X e Y no CRS do projecto;

identificador ou descrição do CRS;

geometria no CRS da camada de destino.

Esta estratégia foi adotada pelo GeoClick Capture e facilita comparação com mapas, GPS e bases institucionais.

10. Leitura de GPX e escrita de formatos GIS

Um ficheiro GPX pode expor subcamadas:

```
waypoints;
routes;
route_points;
tracks;
track_points.
```

Nem todos os ficheiros possuem todas as subcamadas. Ausência não é, por si só, erro.

10.1 Abrir subcamada GPX

```
uri = f"{gpx_path}?type={layer_name}"
source = QgsVectorLayer(uri, layer_name, "gpx")
if not source.isValid() or source.featureCount() == 0:
    # Registrar como missing/empty, não como falha fatal.
    return
```

10.2 Escolher formatos de saída

Uma tabela de configuração reduz condicionais:

```
OUTPUT_FORMATS = {
    "ESRI Shapefile": {
        "driver": "ESRI Shapefile",
        "extension": ".shp",
        "layer_creation_options": ["ENCODING=UTF-8"],
    },
    "GeoPackage": {
        "driver": "GPKG",
        "extension": ".gpkg",
        "layer_creation_options": [],
    },
    "GeoJSON": {
        "driver": "GeoJSON",
        "extension": ".geojson",
        "layer_creation_options": [],
    },
}
```

10.3 Limitações do Shapefile

O Shapefile é amplamente suportado, mas possui limitações:

nomes de campos curtos;

conjunto de ficheiros relacionados;

tipos de dados limitados;

dificuldades de encoding;

um tipo de geometria por camada.

Para dados modernos, GeoPackage é muitas vezes a melhor saída padrão. Contudo, mantenha Shapefile quando a interoperabilidade institucional exigir.

10.4 Proveniência

Ao fundir dados, adicione campos que identifiquem a origem:

source_file

source_path

source_layer

Sem proveniência, a fusão reduz rastreabilidade. O GPX Batch Converter inclui estes campos nos resultados fundidos.

11. Tarefas em segundo plano

Operações longas não devem executar no thread da interface. Um ciclo com centenas de ficheiros pode congelar o QGIS e levar o utilizador a encerrar a aplicação.

11.1 QgsTask

```
class ConversionTask(QgsTask):
```

```
    def __init__(self, files, callback):
```

```
        super().__init__("Convert files", task_can_cancel_flag())
```

```
        self.files = files
```

```
        self.callback = callback
```

```
        self.results = []
```

```
        self.error = None
```

```
    def run(self):
```

```
        try:
```

```
            for index, path in enumerate(self.files, start=1):
```

```
                if self.isCanceled():
```

```
                    return False
```

```
                self.convert_one(path)
```

```
                self.setProgress(index / len(self.files) * 100)
```

```
            return True
```

```
except Exception as exc:
    self.error = exc
    return False
```

```
def finished(self, result):
    self.callback(self, result)
```

Adicionar ao gestor:

```
QgsApplication.taskManager().addTask(task)
```

11.2 Regra do thread principal

No método run() de uma tarefa:

não altere widgets;

não adicione camadas ao projecto;

não aceda a objectos Qt que pertençam ao thread principal;

trabalhe com dados independentes, caminhos e APIs thread-safe.

No finished() ou callback:

actualize a interface;

adicione saídas ao projecto;

apresente o resumo.

11.3 Cancelamento

Cancelamento deve ser cooperativo. Verifique isCanceled() em intervalos curtos. Se existe processo externo activo, termine-o com timeout e depois force a paragem, registando falhas.

No GPX Batch Converter, a tarefa mantém referência ao processo GDAL actual, chama terminate(), aguarda e usa kill() apenas quando necessário.

12. Resultados, progresso e logs

Uma barra de progresso isolada não explica o que aconteceu. Para lotes, mantenha uma lista de resultados com:

```
source_file
layer
status
feature_count
output_path
message
```

Estados recomendados:

```
converted;
```

```
merged;
included;
missing_or_empty;
skipped_existing;
failed;
cancelled.
```

O resumo deve separar falhas reais de camadas ausentes. Isto evita que um GPX sem waypoints seja apresentado como erro.

12.1 Mensagens no QGIS

```
self.iface.messageBar().pushMessage(
    "Plugin",
    "Conversion completed.",
    level=success_level,
    duration=5,
)
```

Para diagnóstico técnico:

```
QgsMessageLog.logMessage(
    detailed_message,
    "My Plugin",
    level=Qgis.MessageLevel.Warning,
)
```

Não mostre dados sensíveis, tokens ou caminhos confidenciais em logs públicos.

13. Ferramentas de mapa

Para responder a cliques:

```
from qgis.gui import QgsMapToolEmitPoint

self.tool = QgsMapToolEmitPoint(self.canvas)
self.tool.canvasClicked.connect(self.handle_map_click)
self.canvas.setMapTool(self.tool)
```

Ao desactivar:

```
if self.canvas.mapTool() is self.tool:
    self.canvas.unsetMapTool(self.tool)
```

13.1 Identificar a feição sob o clique

`QgsMapToolIdentify` pode devolver a camada e a feição. Restrinja a pesquisa a camadas relevantes e visíveis quando possível. Guarde nome, ID da camada e ID da feição para auditoria.

13.2 Estado visual

A acção de captura deve ser verificável. O painel e a toolbar devem mostrar o mesmo estado. Mostre uma mensagem curta quando o modo é activado.

14. Snapping a vértices e segmentos

Snapping melhora precisão e evita pontos quase coincidentes.

Estratégia robusta:

usar a configuração de snapping do projecto;

se não existir correspondência, procurar camadas visíveis de linha e polígono;

dar prioridade ao vértice mais próximo;

usar o segmento mais próximo apenas quando nenhum vértice está dentro da tolerância;

registar se houve snapping, o tipo e a distância.



Fluxo de captura do GeoClick Capture

14.1 Tolerância em pixels

A tolerância em pixels oferece experiência consistente em diferentes escalas. Converta para unidades do mapa:

```
tolerance_map = tolerance_pixels * canvas.mapUnitsPerPixel()
```

Quando a camada tem CRS diferente, transforme o ponto e uma distância de referência para estimar a tolerância na unidade da camada.

14.2 Auditoria do snapping

Campos úteis:

snapped boolean

snap_type vertex | segment | project snapping

snap_distance distância em unidades do mapa

15. Pedidos de rede e geocodificação reversa

Plugins QGIS devem utilizar `QgsNetworkAccessManager` para respeitar proxy, autenticação e definições de rede da aplicação.

```
manager = QgsNetworkAccessManager.instance()
request = QNetworkRequest(QUrl(url))
reply = manager.get(request)
reply.finished.connect(lambda: self.handle_reply(reply))
```

15.1 Regras de um cliente responsável

identificar a aplicação no User-Agent;
 respeitar limites do fornecedor;
 adicionar timeout;
 tratar redireccionamentos com segurança;
 validar HTTP, rede, SSL e JSON;
 armazenar cache;
 evitar pedidos duplicados;
 permitir desligar a funcionalidade;
 manter a função principal independente da rede.

No GeoClick Capture, o ponto é sempre guardado primeiro. A geocodificação actualiza o campo de localização depois. Assim, falha de Internet não causa perda do registo.

15.2 Sem resultado não é falha de rede

Um serviço pode responder que não encontrou endereço. Trate separadamente:

falha de ligação;
 limite HTTP 429;
 rejeição 403;
 erro do servidor;
 resposta válida sem endereço.

Uma estratégia de fallback pode pedir níveis mais amplos: endereço, assentamento, cidade, província e país. Respeite o intervalo mínimo entre pedidos.

16. Preferências persistentes

QgsSettings guarda opções por utilizador:

```
settings = QgsSettings()
settings.setValue("my_plugin/operator", operator_name)
operator_name = settings.value("my_plugin/operator", "", type=str)
```

Use um prefixo exclusivo. Guarde apenas preferências, não dados sensíveis.

Exemplos:

última pasta de saída;
 formato preferido;
 geocodificação ligada/desligada;
 nome do operador;

tolerância de snapping;

camada seleccionada, quando a referência puder ser restaurada com segurança.

Parte IV - Desenvolvimento avançado

17. Compatibilidade QGIS 3/4 e Qt 5/6

A transição para Qt 6 introduziu enums scoped. Código antigo:

```
Qt.RightDockWidgetArea
QMessageBox.Yes
QgsWkbTypes.PointGeometry
```

Código Qt 6:

```
Qt.DockWidgetArea.RightDockWidgetArea
QMessageBox.StandardButton.Yes
QgsWkbTypes.GeometryType.PointGeometry
```

Para suportar ambas as gerações, resolva em tempo de execução:

```
def compat_enum(container, scoped_container, member, legacy):
    scoped = getattr(container, scoped_container, None)
    if scoped is not None:
        return getattr(scoped, member)
    return getattr(container, legacy)
```

```
RIGHT_DOCK_AREA = compat_enum(
    Qt,
    "DockWidgetArea",
    "RightDockWidgetArea",
    "RightDockWidgetArea",
)
```

17.1 Importações

Use sempre:

```
from qgis.PyQt.QtCore import Qt
from qgis.PyQt.QtWidgets import QAction
```

Evite importar directamente de PyQt5 ou PyQt6. O módulo qgis.PyQt selecciona a implementação fornecida pelo QGIS.

17.2 Testes estáticos de enums

Um teste simples pode analisar AST e proibir acessos antigos no código executável. AST é superior à procura textual porque comentários podem mencionar nomes antigos sem representar erro.

18. Segurança

18.1 Subprocessos

Quando um plugin chama GDAL ou outra ferramenta:

localize executáveis confiáveis;

resolva caminhos absolutos;

valide cada argumento;

passe argumentos numa lista ou tuplo;

use `shell=False`;

rejeite bytes nulos;

não construa comandos com concatenação de entrada do utilizador;

limite o ambiente quando necessário;

capture stdout, stderr e código de saída.

```
process = subprocess.Popen(
    validated_command,
    shell=False,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True,
    encoding="utf-8",
    errors="replace",
)
```

No GPX Batch Converter, apenas os caminhos absolutos detectados para ogr2ogr e ogrinfo são permitidos.

18.2 SQL, expressões e nomes

Evite SQL montado dinamicamente com nomes de ficheiros. Use APIs OGR/PyQGIS, parâmetros e identificadores validados. Normalize nomes de ficheiro:

```
def clean_filename(name, fallback="output"):
    cleaned = re.sub(r"^[^\\w\\s-]", "", str(name), flags=re.UNICODE)
    cleaned = re.sub(r"[-\\s]+", "_", cleaned).strip("_")
    return cleaned or fallback
```

18.3 Rede

HTTPS;

redireccionamentos limitados;

sem tokens em URL ou log;

timeout;

validação de resposta;

cache;

política de privacidade documentada.

18.4 Dados locais

Não apague ou sobrescreva sem consentimento claro. Para Shapefile, considere todos os componentes. Quando o output está aberto no QGIS, a escrita pode falhar por bloqueio; apresente instruções úteis.

19. Testes

19.1 Pirâmide prática

Funções puras: nomes, validação, extensão, cache key;

Testes de metadata e pacote;

Testes estáticos: enums, caminhos, imports proibidos;

Testes PyQGIS: camadas, transformações, escrita;

Testes manuais: interface, mapa, rede e plataformas.

19.2 unittest

```
import unittest
```

```
from my_plugin.utils import clean_filename
```

```
class FilenameTests(unittest.TestCase):
```

```
    def test_removes_unsupported_characters(self):
        self.assertEqual(clean_filename("A/B:C"), "ABC")
```

```
    def test_uses_fallback(self):
        self.assertEqual(clean_filename("!!!"), "output")
```

19.3 Metadata

```
import configparser
```

```
from pathlib import Path
```

```
PLUGIN = Path(__file__).parents[1] / "my_plugin"
```

```
parser = configparser.ConfigParser()
```

```
parser.read(PLUGIN / "metadata.txt", encoding="utf-8")
```

```
assert parser["general"]["version"] == (PLUGIN / "VERSION").read_text().strip()
```

19.4 Testes do ZIP

Verifique:

um único directório de topo;

ficheiros obrigatórios;

sem `__pycache__`, `.pyc`, `.git`, `__MACOSX`;

metadata sincronizada;

todos os recursos referenciados existem;

tamanho aceitável.

20. Empacotamento

Estrutura correcta:

```
my_plugin-1.0.0.zip
└─ my_plugin/
    ├── __init__.py
    ├── metadata.txt
    ├── plugin.py
    ├── LICENSE
    └─ ...
```

Estrutura incorrecta:

```
my_plugin-1.0.0.zip
├─ README.md
└─ src/
    └─ my_plugin/
```

Comando:

```
zip -qr my_plugin-1.0.0.zip my_plugin \
-x '*/__pycache__/*' '*.pyc' '.git/*'
```

21. Git e fluxo de colaboração

Fluxo recomendado:

```
main
└─ feature/snapping
    └─ Pull Request -> CI -> review -> merge
```

Commits devem descrever uma unidade lógica:

Add cancellable background conversion

Fix Qt 6 dock widget enum compatibility

Validate GDAL executable paths

Evite commits como update, changes ou final final.

21.1 Pull Request

Inclua:

o que mudou;

motivo;

impacto no utilizador;

risco e compatibilidade;

testes realizados;

capturas de ecrã, quando a interface mudou.

22. Integração contínua e releases



Pipeline de release

Um workflow mínimo:

name: Plugin checks

on:

push:

pull_request:

jobs:

validate:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v4

- **uses:** actions/setup-python@v5

with:

python-version: "3.12"

- **run:** python -m compileall -q my_plugin

- **run:** python -m unittest discover -s tests -v

- **run:** |

test -f my_plugin/metadata.txt

test -f my_plugin/__init__.py

test -f my_plugin/LICENSE

- **run:** zip -qr my_plugin.zip my_plugin -x '*/__pycache__/*' '*.pyc'

Uma release profissional:

valida que a tag coincide com VERSION e metadata.txt;

executa testes;

gera o ZIP a partir da tag;

- publica notas de versão;
- anexa o pacote;
- mantém o código da release idêntico ao repositório indicado na metadata.

GeoClick Capture adoptou este processo na versão 1.2.6.

23. Publicação no repositório oficial do QGIS

Antes do upload:

- obtenha OSGEO ID;
- confirme links de homepage, repositório e tracker;
- use licença compatível;
- inclua descrição curta em inglês;
- documente dependências;
- não inclua executáveis ou bibliotecas compiladas;
- mantenha o ZIP dentro do limite;
- verifique duplicação funcional;
- teste Windows, Linux e macOS quando possível;
- mantenha changelog e versão actualizados.

23.1 Aprovação

Novos plugins passam por validação automatizada e revisão. O revisor pode instalar uma amostra aleatória e verificar se o QGIS inicia sem falhar. Um traceback em `initGui()` é motivo suficiente para bloqueio.

23.2 Actualizações

Para uma nova versão:

- aumente a versão;
- actualize changelog;
- confirme links;
- execute todos os testes;
- gere o ZIP a partir do mesmo commit da release;
- carregue como nova versão do mesmo plugin.

Não altere o nome apenas por suportar uma versão nova de QGIS.

24. Tradução, acessibilidade e documentação

24.1 Internacionalização

Use inglês como idioma base quando pretende colaboração internacional. Prepare strings com `self.tr()` e ficheiros `.ts/.qm` para traduções.

Evite concatenar frases que dificultam tradução:

Fraco

```
message = "Converted " + str(count) + " files"
```

Melhor

```
message = self.tr("Converted {count} files").format(count=count)
```

24.2 Acessibilidade

textos claros nos botões;

tooltips;

ordem de tabulação lógica;

não depender apenas da cor;

ícones acompanhados de texto nos contextos principais;

mensagens legíveis e copiáveis;

atalhos configuráveis quando apropriado.

24.3 Documentação mínima

README:

problema e solução;

funcionalidades;

instalação;

uso;

compatibilidade;

limitações;

desenvolvimento;

como reportar erros;

licença.

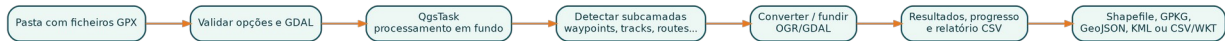
Inclua dados mínimos seguros quando ajudam a reproduzir o fluxo.

Parte V - Estudos de caso

25. GPX Batch Converter

Repositório: <https://github.com/Jubilio/gpx-batch-converter>

Plugin oficial: https://plugins.qgis.org/plugins/gpx_batch_converter/



Fluxo do GPX Batch Converter

25.1 Problema original

Converter centenas de ficheiros GPX manualmente, camada por camada, cria atrasos, nomes inconsistentes e perda de rastreabilidade. A primeira versão automatizou a conversão de waypoints, routes, route_points, tracks e track_points.

25.2 Arquitectura

```

gpx_batch_converter/
├── __init__.py      -> classFactory()
├── plugin.py        -> menu, toolbar e ciclo de vida
├── dialog.py        -> interface, validação e apresentação
├── conversion_task.py -> QgsTask, GDAL, cancelamento e resultados
├── metadata.txt
├── icon.png
├── LICENSE
└── README.md
  
```

Esta separação permite testar funções de conversão sem instanciar toda a interface.

25.3 Evolução funcional

Fase 1 - lote básico

pastas de entrada e saída;
subcamadas seleccionáveis;
overwrite;
adicionar resultados ao projecto;
progresso e resumo.

Fase 2 - fusão e proveniência

um output por tipo de camada;
prefixo configurável;
campos source_file, source_path e source_layer;

camadas ausentes separadas de erros.

Fase 3 - produto profissional

Shapefile, GeoPackage, GeoJSON, KML e CSV/WKT;

painel de resultados;

relatório CSV;

QGIS Task Manager;

cancelamento;

interface responsiva.

Fase 4 - segurança e publicação

caminhos absolutos de GDAL;

shell=False;

argumentos validados;

remoção de SQL dinâmico;

metadata completa;

compatibilidade QGIS 3.28-4.99 e Qt 6;

aprovação no repositório oficial.

25.4 Lições

Diferencie ausência esperada de falha.

Uma operação demorada precisa de tarefa, progresso e cancelamento.

Dados fundidos precisam de proveniência.

Processos externos exigem validação de segurança.

Relatórios estruturados são mais úteis que uma caixa final.

26. GeoClick Capture

Repositório: <https://github.com/Jubilio/qgis-latlon>

Release usada no estudo: v1.2.6.

26.1 Problema e reposicionamento

A primeira ideia concentrava-se em coordenadas, uma área já coberta por plugins maduros. O projecto foi redefinido como uma ferramenta de log auditável para verificação de campo, revisão cartográfica e controlo de qualidade GIS.

Essa decisão ilustra um princípio importante: uma boa alteração de produto pode ser mais valiosa que adicionar funcionalidades.

26.2 Registo de auditoria

Cada captura pode armazenar:

session_id
 captured_at
 operator
 category
 status
 note
 lat / lon
 map_x / map_y
 project_crs
 project_name
 map_scale
 source_layer
 source_layer_id
 source_feature_id
 location
 snapped
 snap_type
 snap_distance

26.3 Evolução

Base

clique no mapa;
 WGS 84;
 camada temporária;
 exportação.

Painel Capture Log

sessão;
 operador, categoria, estado e nota;
 QgsMapLayerComboBox;
 tabela de registos;
 undo, delete e clear;
 CSV, GeoJSON e GeoPackage;
 preferências persistentes.

Precisão espacial

snapping do projecto;
 fallback automático para vértices;

fallback para segmentos;
 tolerância em pixels;
 campos de auditoria.

Rede

QgsNetworkAccessManager;
 cache;
 um pedido por segundo;
 timeout;
 redireccionamentos seguros;
 diagnóstico HTTP/SSL/DNS/proxy;
 fallback de endereço para área administrativa;
 captura independente da rede.

Compatibilidade e qualidade

enums Qt 6 scoped;
 fallbacks Qt 5;
 testes AST;
 ícones SVG específicos;
 GitHub Actions e release automatizada.

26.4 Lições

O CRS do mapa não é necessariamente WGS 84.

Interface persistente favorece QDockWidget.

Snapping deve ser previsível e auditável.

Serviço externo não deve comprometer a função principal.

Validadores podem analisar ficheiros que não são executados directamente; mantenha todo o pacote compatível.

27. Comparação dos dois padrões

Os dois plugins partilham o mesmo ciclo de vida QGIS, metadata, disciplina de empacotamento e preocupação com Qt 6. A diferença está no centro da arquitectura: o GPX Batch Converter organiza uma operação de lote; o GeoClick Capture mantém uma sessão interactiva no mapa. A tabela seguinte resume as decisões principais.

Interacção principal

GPX Batch Converter: ficheiros e operação de lote.

GeoClick Capture: clique no mapa e sessão interactiva.

Interface

GPX Batch Converter: QDialog, adequado a uma tarefa com início e fim definidos.

GeoClick Capture: QDockWidget, adequado a trabalho contínuo enquanto o mapa permanece visível.

Processamento e dependências

GPX Batch Converter: usa QgsTask para operações longas e integra ferramentas GDAL/OGR disponibilizadas pelo ambiente QGIS.

GeoClick Capture: executa sobretudo interacções curtas; a dependência externa principal é a geocodificação reversa opcional.

Geometria e estado

GPX Batch Converter: lê, converte e combina geometrias, mantendo opções e resultados do lote.

GeoClick Capture: cria pontos, transforma CRS, aplica snapping e mantém sessão, camada e preferências.

Saídas e risco central

GPX Batch Converter: produz múltiplos formatos; os principais riscos são bloqueio da interface, subprocessos e tratamento incompleto de lotes.

GeoClick Capture: produz uma camada auditável e exportações; os principais riscos são CRS, snapping, rede e compatibilidade Qt.

A arquitectura ideal depende do tipo de problema. Não copie uma estrutura inteira sem compreender as necessidades.

Parte VI - Projecto práctico

28. Construir o Quick Point Logger

O repositório deste manual inclui `examples/quick_point_logger`, um plugin pedagógico que combina:

- acção verificável;
- ferramenta de clique;
- camada de memória;
- transformação para WGS 84;
- painel simples;
- exportação GeoJSON;
- definições persistentes;

compatibilidade básica Qt 5/6.

28.1 Passo 1 - criar a pasta

```
quick_point_logger/
├── __init__.py
├── metadata.txt
├── plugin.py
├── dock.py
├── utils.py
├── icons/capture.svg
├── LICENSE
└── README.md
```

28.2 Passo 2 - metadata e entrada

Implemente metadata.txt e classFactory() com a estrutura mostrada nos capítulos 4 e 5.

28.3 Passo 3 - acção e ferramenta

Crie uma acção verificável, associe ao menu Vector e à toolbar. Quando activada, defina QgsMapToolEmitPoint no canvas.

28.4 Passo 4 - camada e campos

Crie a camada WGS 84 e adicione campos de tempo, coordenadas e nota. Transforme o clique antes de escrever.

28.5 Passo 5 - painel

Inclua:

nota para próxima captura;

número de pontos;

iniciar/parar;

undo;

exportar.

28.6 Passo 6 - testes

Teste:

metadata;

versão;

estrutura;

função DMS;

nome de ficheiro;

ausência de imports directos PyQt5/PyQt6.

28.7 Passo 7 - pacote

```
python -m compileall -q quick_point_logger
python -m unittest discover -s tests -v
zip -qr quick_point_logger-0.1.0.zip quick_point_logger \
-x '*/__pycache__/*' '*.pyc'
```

29. Exercícios graduais

Básico

Alterar nome, ícone e mensagem da acção.

Adicionar um campo operator.

Guardar a última nota com QgsSettings.

Validar se a camada está activa.

Intermédio

Permitir seleccionar camada de destino com QgsMapLayerComboBox.

Adicionar exportação GeoPackage.

Implementar undo da última feição.

Adicionar snapping do projecto.

Criar tabela de registos.

Avançado

Executar exportação pesada em QgsTask.

Adicionar geocodificação reversa assíncrona com cache.

Criar testes AST para enums Qt.

Adicionar tradução português/inglês.

Criar release automática por tag.

Preparar submissão ao repositório oficial.

30. Diagnóstico de erros

O diagnóstico deve começar pelo ciclo de vida: confirmar se o pacote foi descoberto, se classFactory() importou a classe, se initGui() terminou e se a operação específica recebeu entradas válidas. Depois consulte **Log Messages**, a consola Python e o traceback completo. Os padrões abaixo cobrem as falhas mais comuns.

Plugin não aparece: metadata inválida ou pasta instalada no nível errado. Verifique metadata.txt, o nome da pasta e a localização no perfil.

Erro em classFactory(): import relativo, nome da classe ou dependência indisponível. Teste o import na consola Python do QGIS.

Erro em initGui(): enum Qt, recurso ausente, caminho de ícone ou widget incompatível. Valide todos os ficheiros do pacote no QGIS 3 e 4.

Interface congela: processamento pesado executado no thread principal. Mova o lote para QgsTask e actualize widgets apenas no callback final.

Coordenadas erradas: ponto do canvas tratado directamente como longitude/latitude. Transforme do CRS do projecto para EPSG:4326.

Output bloqueado: camada aberta no QGIS ou ficheiro utilizado por outra aplicação. Remova a camada, feche o programa e repita.

Snapping não ocorre: configuração, tolerância, visibilidade ou transformação de CRS incorrecta. Teste o snapping do projecto e o fallback separadamente.

Rede falha apenas no QGIS: pedido feito com biblioteca que ignora o proxy da aplicação. Use QgsNetworkAccessManager.

QGIS fecha ou produz erro ao desactivar: tarefa, reply ou map tool continuam activos. Cancele, aborte e remova tudo em unload().

ZIP rejeitado: directório de topo, metadata, licença, versão ou ficheiros de cache incorrectos. Execute a checklist de publicação.

Validador Qt 6 falha: enum antigo ou import directo de PyQt5. Use enums scoped e qgis.PyQt.

Processo externo não cancela: subprocesso executado sem polling. Mantenha referência ao processo, use timeout e force a paragem apenas quando necessário.

Resultados inconsistentes: estados ausentes ou excepções agregadas numa mensagem genérica. Registe uma linha de resultado por ficheiro e subcamada.

Camada não recebe campos: provider não suporta alteração ou camada está em estado incompatível. Verifique capacidades e actualize os campos.

Erro só em determinados projectos: CRS inválido, camada corrompida, geometria diferente ou definições persistentes antigas. Registe o contexto e teste com projecto vazio.

31. Checklists

31.1 Antes de desenvolver

- ☐ problema e utilizador definidos;
- ☐ plugins semelhantes analisados;
- ☐ escopo da primeira versão limitado;
- ☐ versões QGIS escolhidas;

- ☐ dados mínimos disponíveis;
- ☐ riscos de rede, ficheiros e segurança identificados.

31.2 Antes de criar PR

- ☐ código compila;
- ☐ testes passam;
- ☐ plugin carrega e descarrega;
- ☐ fluxo principal testado;
- ☐ erros e cancelamento testados;
- ☐ documentação actualizada;
- ☐ sem ficheiros gerados ou cache;
- ☐ screenshots incluídos se necessário.

31.3 Antes de publicar

- ☐ VERSION, metadata e changelog sincronizados;
- ☐ homepage, repository e tracker funcionam;
- ☐ licença incluída no pacote;
- ☐ descrição em inglês;
- ☐ dependências documentadas;
- ☐ QGIS 3/4 e plataformas testadas conforme compromisso;
- ☐ ZIP com uma pasta de topo;
- ☐ sem binários, .git, __pycache__ e .pyc;
- ☐ pacote idêntico ao commit da release;
- ☐ dados de teste não contêm informação sensível.

32. Próximos níveis

Depois de dominar estes padrões, explore:

Processing providers e algoritmos personalizados;

QGIS Server plugins;

formulários e widgets de edição;

layouts e relatórios;

integração com bases PostGIS;

autenticação do QGIS;

modelos de dados e validação de topologia;

testes em containers QGIS;

qgis-plugin-ci;

publicação de documentação com MkDocs ou Sphinx.

A evolução deve ser guiada por problemas reais, não por uma lista de funcionalidades. Os dois plugins estudados tornaram-se mais fortes quando cada versão respondeu a falhas observadas: lentidão, falta de auditoria, incompatibilidade Qt 6, snapping, diagnósticos de rede, validação de segurança e publicação reproduzível.

Apêndice A - Template de metadata

[general]

name=My QGIS Tool

description=One-line English description.

about=Detailed explanation, dependencies and limitations.

version=0.1.0

qgisMinimumVersion=3.28

qgisMaximumVersion=4.99

author=Author Name

email=author@example.org

homepage=<https://github.com/user/repo#readme>

repository=<https://github.com/user/repo>

tracker=<https://github.com/user/repo/issues>

license=MIT

category=Vector

tags=meaningful,searchable,tags

experimental=**True**

deprecated=**False**

hasProcessingProvider=**False**

supportsQt6=**True**

icon=icons/icon.svg

changelog=0.1.0 - Initial release.

Apêndice B - Template de unload()

```
def unload(self):
```

```
    if self.tool is not None and self.canvas.mapTool() is self.tool:
        self.canvas.unsetMapTool(self.tool)
```

```
    if self.task is not None:
        self.task.cancel()
        self.task = None
```

```
    for reply in list(self.pending_replies):
        try:
```



```

        reply.abort()
        reply.deleteLater()
    except RuntimeError:
        pass
    self.pending_replies.clear()

    if self.action is not None:
        self.iface.removePluginVectorMenu(self.MENU, self.action)
        self.iface.removeToolBarIcon(self.action)
        self.action.deleteLater()
        self.action = None

    if self.dock is not None:
        self.iface.removeDockWidget(self.dock)
        self.dock.deleteLater()
        self.dock = None

```

Apêndice C - Estrutura de resultado de lote

```

result = {
    "source_file": str(source_path),
    "layer": layer_name,
    "status": "converted",
    "feature_count": count,
    "output": str(output_path),
    "message": "",
}

```

Apêndice D - Glossário

API - interface utilizada por código para interagir com uma aplicação ou serviço.

Canvas - área de mapa do QGIS.

CRS - sistema de referência de coordenadas.

GDAL/OGR - bibliotecas e ferramentas para leitura, transformação e escrita de dados geoespaciais.

GeoPackage - formato SQLite aberto para dados geográficos.

Metadata - dados que descrevem o plugin, incluindo nome, versão e compatibilidade.

Provider - componente QGIS que fornece acesso a uma fonte de dados.

PyQGIS - API Python do QGIS.

Qt - framework de interface utilizado pelo QGIS.

Signal/slot - mecanismo Qt para ligar eventos a funções.

Snapping - ajustamento de uma coordenada a um vértice ou segmento próximo.

Thread principal - thread responsável pela interface gráfica.

Referências e recursos

QGIS Documentation. *Developing Python Plugins*.

https://docs.qgis.org/4.2/en/docs/pyqgis_developer_cookbook/plugins/index.html

QGIS Documentation. *Structuring Python Plugins*.

https://docs.qgis.org/4.2/en/docs/pyqgis_developer_cookbook/plugins/plugins.html

QGIS Documentation. *Releasing your plugin*.

https://docs.qgis.org/4.2/en/docs/pyqgis_developer_cookbook/plugins/releasing.html

QGIS Plugins. *Publishing guidelines*. <https://plugins.qgis.org/docs/publish>

QGIS API Documentation. <https://api.qgis.org/>

GPX Batch Converter. <https://github.com/Jubilio/gpx-batch-converter>

GeoClick Capture. <https://github.com/Jubilio/qgis-latlon>

GPX Batch Converter no repositório oficial.

https://plugins.qgis.org/plugins/gpx_batch_converter/

Nota de licenciamento

O texto do manual é disponibilizado sob **Creative Commons Attribution 4.0 International (CC BY 4.0)**. Os exemplos de código incluídos no repositório são disponibilizados sob **MIT License**. As APIs, nomes e marcas QGIS pertencem aos respectivos projectos e titulares.